A Nominal Formalization of the Confluence of a Calculus with Explicit Substitutions via the Z Property

Flávio L. C. de Moura[1 [0000−0002−9390−5751]]

Departamento de Ciência da Computação
Universidade de Brasília, Brasília, Brazil
flaviomoura@unb.br

Abstract. The Z property is a compact proof technique for confluence that has been used to give elegant confluence proofs for the λ-calculus and several of its extensions. Previous formalizations of the (compositional) Z property, as well as of the Substitution Lemma for calculi with explicit substitutions, were developed in Rocq by the author. In this work, we formalize the λ_x-calculus, an extension of the λ-calculus with explicit substitutions, and we apply our formalization of the compositional Z property to give a mechanized proof of confluence for the λ_x-calculus. To the best of our knowledge, this is the first formalized confluence proof via the compositional Z property. The formalization is done in the Rocq Prover (version 9.1.1) and is freely available (zenodo???). It comprises around 3 500 lines of Rocq code and 150 lemmas and theorems, and provides a reusable nominal toolkit for confluence proofs of calculi with explicit substitutions.

Keywords: Nominal syntax, Confluence, Z property, Explicit substitutions, Rocq Prover, Formal verification.

1 Introduction

We present a framework for studying properties of extensions of the λ-calculus [3], based on the nominal approach [15]. In the nominal setting, variables are represented by atoms, structureless entities with a decidable equality. Therefore, different names denote different atoms, and hence different variables. The nominal approach is close to the usual paper-and-pencil notation of the λ-calculus, whose grammar of terms is given by:

$$t ::= x \mid \lambda_x.t \mid t\ t \tag{1}$$

where x represents a variable taken from an enumerable set, $\lambda_x.t$ is an abstraction, and $t\ t$ is an application. The abstraction is the only binding operator: in the expression $\lambda_x.t$, x binds in t, called the scope of the abstraction. This means that every free occurrence of x in t becomes bound in $\lambda_x.t$. An occurrence of a

variable that is not bound is free. Each occurrence of a variable is either bound or free, but the same variable can occur both bound and free in a term, as in $(\lambda_y.y)\ y$. The main rule of the λ-calculus, named β-reduction, is given by:

$$(\lambda_x.t)\ u \rightarrow_\beta \{x := u\}t \tag{2}$$

where $\{x := u\}t$ represents the result of substituting all free occurrences of the variable x in t with u, renaming bound variables whenever needed to avoid capturing free variables of u. We call t the body of the metasubstitution, and u its argument. In other words, $\{x := u\}t$ is a metanotation for a capture-free substitution. For instance, the λ-term $(\lambda_x\lambda_y.x\ y)\ y$ has both bound and free occurrences of the variable y, and in order to β-reduce it, one has to substitute the free variable y for all free occurrences of the variable x in the term $(\lambda_y.x\ y)$. A naive substitution would capture the free variable y, *i.e.*, the free occurrence of y before the β-reduction would become bound after the β-reduction step. A renaming of bound variables avoids such a capture: in this example, one can take an α-equivalent[1] term, say $(\lambda_z.x\ z)$, and perform the β-step correctly as $(\lambda_x\lambda_y.x\ y)\ y \rightarrow_\beta \lambda_z.y\ z$. In the nominal setting, variables are renamed via name swapping, which is defined on variables as follows:

$$(x\ y)z := \begin{cases} y, & \text{if } z = x; \\ x, & \text{if } z = y; \\ z, & \text{otherwise.} \end{cases}$$

This notion can be extended to the level of terms in a straightforward way:

$$(x\ y)t := \begin{cases} (x\ y)z, & \text{if } t = z; \\ \lambda_{(x\ y)z}.(x\ y)t_1, & \text{if } t = \lambda_z.t_1; \\ (x\ y)t_1\ (x\ y)t_2, & \text{if } t = t_1\ t_2 \end{cases} \tag{3}$$

In the previous example, the variable capture can be avoided by applying a swap to the body of the abstraction before performing the metasubstitution: $(\lambda_x\lambda_y.x\ y)\ y \rightarrow_\beta \{x := y\}((y\ z)(\lambda_y.x\ y)) = \{x := y\}(\lambda_z.x\ z) = \lambda_z.y\ z$. The metasubstitution can be defined in different ways; in our formalization, it is defined as follows:

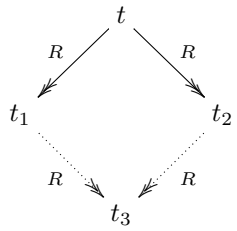
$$\{x := u\}t = \begin{cases} u, & \text{if } t = x; \\ t, & \text{if } t = y \text{ and } x \neq y; \\ \{x := u\}t_1\ \{x := u\}t_2, & \text{if } t = t_1\ t_2; \\ \lambda_x.t_1, & \text{if } t = \lambda_x.t_1; \\ \lambda_z.(\{x := u\}((y\ z)t_1)), & \text{if } t = \lambda_y.t_1, x \neq y \\ & \text{and} \\ & z \notin fv(t\ u) \cup \{x\}. \end{cases} \tag{4}$$

where $fv(t)$ denotes the set of free variables of t. The last case of the definition of metasubstitution is the one that avoids variable capture by applying a swap to the body of the abstraction before performing the metasubstitution. In this case, z is a fresh variable, that is, a variable different from x that does not occur

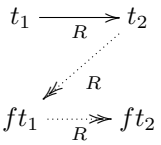
[1] A formal definition of this notion will be given later in this section.

free in t or in u, and it can be chosen arbitrarily. In our formalization, it is chosen by the function *atom_fresh*.

A reduction relation R is confluent (Confl) if any two reductions starting from the same term can be joined, that is, if $t \longrightarrow\!\!\!\!\!\rightarrow_R t_1$ and $t \longrightarrow\!\!\!\!\!\rightarrow_R t_2$, then there exists a term t_3 such that $t_1 \longrightarrow\!\!\!\!\!\rightarrow_R t_3$ and $t_2 \longrightarrow\!\!\!\!\!\rightarrow_R t_3$, as in the following diagram, where $\longrightarrow\!\!\!\!\!\rightarrow_R$ is the reflexive-transitive closure of R and the dotted arrows are existentially quantified:



The Z property [28,12] is a sufficient condition for confluence. Informally, a function f has the Z property for a relation R if, whenever $t_1 \rightarrow_R t_2$, we have $t_2 \longrightarrow\!\!\!\!\!\rightarrow_R f\,t_1 \longrightarrow\!\!\!\!\!\rightarrow_R f\,t_2$, as in the following diagram:



The compositional Z property [25] (Z_comp_aeq in our formalization, modulo α-equivalence) splits this task for a relation $R = R_1 \cup R_2$ and a function $f = f_2 \circ f_1$: it suffices to show that f_1 has the Z property for R_1, that $a \longrightarrow\!\!\!\!\!\rightarrow_R f_2(a)$ for every a in the image of f_1, and that f has a weaker version of the Z property for R_2, namely that $t_1 \rightarrow_{R_2} t_2$ implies $t_2 \longrightarrow\!\!\!\!\!\rightarrow_R f(t_1) \longrightarrow\!\!\!\!\!\rightarrow_R f(t_2)$. Then f has the Z property for R, and hence R is confluent.

The main contributions of this work are as follows:

- a formalization of the (compositional) Z property modulo α-equivalence for nominal terms. The generic formalization in [11] states the Z property and the compositional Z property for an arbitrary relation over an abstract set, and proves that they imply confluence. We instantiate this set with nominal terms with explicit substitutions, and adapt the definitions so that the functions involved are compatible with α-equivalence.
- a toolkit of properties of the metasubstitution on nominal terms, built around the Substitution Lemma for calculi with explicit substitutions formalized in [20] (m_subst_lemma). The Substitution Lemma is the key ingredient to show that the complete development function B commutes with the metasubstitution up to β-reduction (Lemma pure_m_subst_refltrans_beta_B), which is the most delicate step of the confluence proof.
- a formalization of the confluence of the λ_x-calculus that combines the two previous contributions. The reduction $\rightarrow_{lx}$ is split into the b-reduction, which creates explicit substitutions, and the x-reduction, which propagates and eliminates them. The function P, which eliminates all explicit substitutions,

and the function B are shown to satisfy the compositional Z property modulo α-equivalence (lambda_x_Z_comp_aeq), where the Substitution Lemma is used to simulate b-steps by β-steps. The theorems of the first contribution then give the confluence of $\rightarrow_{lx}$ (lambda_x_is_confluent).

To the best of our knowledge, this is the first nominal formalization of the confluence of the λ_x-calculus via the compositional Z property. The rest of the paper is organized as follows. In Subsection 1.1, we extend the syntax with an explicit substitution operator, and define α-equivalence and the metasubstitution on the resulting terms, together with their main properties. In Section 2, we present the λ_x-calculus and prove its confluence via the compositional Z property. In Section 3, we discuss related work, conclude, and outline future work.

1.1 An Explicit Substitution Operator

In this subsection, we extend the grammar (1) with a new constructor called an explicit substitution operator:

$$t ::= x \mid \lambda_x.t \mid t\ t \mid [x := u]t \tag{5}$$

where $[x := u]t$ represents a term with a pending substitution that will be evaluated by specific rules of a substitution calculus. The explicit substitution is intended to simulate the metasubstitution. This formalization aims to be a generic framework applicable to any calculus with explicit substitutions using a named notation for variables. Therefore, we do not fix the rules that simulate the metasubstitution. Designing such rules is not a trivial task, since one can easily lose important properties of the original λ-calculus in the process [23,16].

Calculi with explicit substitutions are formalisms that deconstruct the metasubstitution operation into finer-grained steps, thereby functioning as an intermediary between the λ-calculus and its practical implementations: they shed light on the execution models of higher-order languages. In fact, developing a calculus with explicit substitutions that is faithful to the λ-calculus, in the sense of preserving some of its desired properties, has been the main motivation behind the long list of calculi with explicit substitutions invented over the last decades [1,30,4,9,24,18,6,10,19].

The following inductive definition corresponds to the grammar (5), where the explicit substitution constructor, named *n_sub*, has a special notation. Instead of writing *n_sub t x u*, we write $[x := u]\ t$, as in (5). Accordingly, *n_sexp* denotes the set of nominal λ-expressions equipped with an explicit substitution operator, which we simply call "terms". The Rocq logo on each code snippet links to the corresponding definition in the source repository.

ROCQ

```
Inductive n_sexp : Set :=
| n_var (x:atom)
| n_abs (x:atom) (t:n_sexp)
| n_app (t1:n_sexp) (t2:n_sexp)
```

```
| n_sub (t1:n_sexp) (x:atom) (t2:n_sexp).
```

A term is *pure* if it has no explicit substitution operator. The *size* of a term and its set of free variables *fv_nom* are defined as usual.

In the λ-calculus, terms that differ only in the names of their bound variables are considered equivalent. This equivalence is called α-equivalence, and it is defined by the following inference rules (cf. [31]):

$$(aeq_var)\ \frac{}{x =_\alpha x} \qquad \frac{t_1 =_\alpha t_2}{\lambda_x.t_1 =_\alpha \lambda_x.t_2}\ (aeq_abs_same)$$

$$\frac{x \neq y \qquad x \notin fv(t_2) \qquad t_1 =_\alpha (y\ x)t_2}{\lambda_x.t_1 =_\alpha \lambda_y.t_2}\ (aeq_abs_diff)$$

$$\frac{t_1 =_\alpha t_1' \qquad t_2 =_\alpha t_2'}{t_1\ t_2 =_\alpha t_1'\ t_2'}\ (aeq_app)$$

In [20], this definition was extended to explicit substitutions by adding the following rules:

$$\frac{t_1 =_\alpha t_1' \qquad t_2 =_\alpha t_2'}{[x := t_2]t_1 =_\alpha [x := t_2']t_1'}\ (aeq_sub_same)$$

$$\frac{t_2 =_\alpha t_2' \qquad x \neq y \qquad x \notin fv(t_1') \qquad t_1 =_\alpha (y\ x)t_1'}{[x := t_2]t_1 =_\alpha [y := t_2']t_1'}\ (aeq_sub_diff)$$

In the formalization, the resulting relation is called aeq. We prove its basic properties, namely reflexivity (aeq_refl), symmetry (aeq_sym) and transitivity (aeq_trans), as well as the facts that α-equivalence is stable under swapping (aeq_swap) and that α-equivalent terms have the same free variables (aeq_fv_nom). The next lemma shows how the metasubstitution propagates through an abstraction whose bound variable y is different from the variable x being replaced: up to α-equivalence, the bound variable is renamed to a fresh variable z. This renaming prevents the free variables of u from being captured by the abstraction:

ROCQ

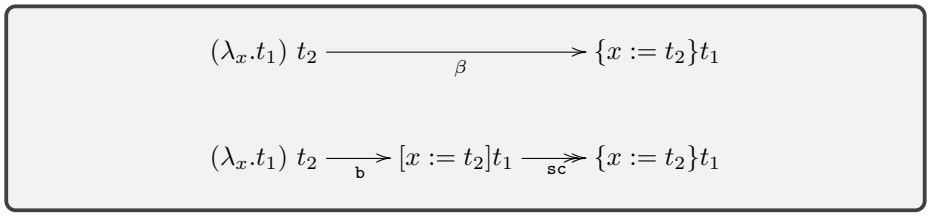
```
Lemma aeq_m_subst_abs_neq: ∀ t u x y z, x ≠ y → z ∉ fv_nom u ∪ fv_nom
(n_abs y t) ∪ {{x}} → {x := u}(n_abs y t) =α n_abs z ({x := u}(swap y z t)).
```

2 The λ_x-Calculus with Explicit Substitutions

In this section, we formalize a version of the λ_x-calculus [21,22,29,5] in a nominal framework and prove its confluence via the compositional Z property [25]. This

task can be seen as an application of our formalizations of the compositional Z property [11] and of the extension of the Substitution Lemma for calculi with explicit substitutions [20].

As discussed in Subsection 1.1, calculi with explicit substitutions deconstruct β-reduction into finer-grained steps. In the λ_x-calculus, a β-step is divided into two parts, one that initiates its simulation and another that completes it, as illustrated by the following figure:



where the `b` step initiates the simulation and the `sc` steps complete it. Here, the metasubstitution is extended to terms with explicit substitutions as follows:

$$\{x := u\}t = \begin{cases} u, & \text{if } t = x; \\ t, & \text{if } t = y \text{ and } x \neq y; \\ \{x := u\}t_1\ \{x := u\}t_2, & \text{if } t = t_1\ t_2; \\ \lambda_x.t_1, & \text{if } t = \lambda_x.t_1; \\ \lambda_z.(\{x := u\}((y\ z)t_1)), & \text{if } t = \lambda_y.t_1, x \neq y \\ & \text{and} \\ & z \notin fv(t\ u) \cup \{x\}; \\ [x := \{x := u\}t_2]t_1, & \text{if } t = [x := t_2]t_1; \\ [z := \{x := u\}t_2]\{x := u\}((y\ z)t_1), & \text{if } t = [y := t_2]t_1, \\ & x \neq y \text{ and} \\ & z \notin fv(t\ u) \cup \{x\}. \end{cases} \tag{6}$$

The `b` step is inductively defined as the reduction of a β-redex $(\lambda_x.t_1)\ t_2$, written (*n_app* (*n_abs x t1*) *t2*) in Rocq, to an explicit substitution:

```
Inductive betax : Rel n_sexp :=
 | step_betax : ∀ (t1 t2: n_sexp) (x: atom),
      betax (n_app (n_abs x t1) t2) ([x:=t2]t1).
```

This reduction is then performed modulo α-equivalence (cf. [20]).

We write $\to_b$ for the contextual closure of the b-step modulo α-equivalence, and $\to_x$ for the contextual closure of the reduction rules of the substitution calculus that are given below, where $x \neq y$:

$$
\begin{array}{lll}
[y := t]y & \to_{var} & t \\
[y := t]x & \to_{gc} & x \\
[y := t_2](\lambda_y.t_1) & \to_{abs1} & \lambda_y.t_1 \\
[y := t_2](\lambda_x.t_1) & \to_{abs2} & \lambda_x.([y := t_2]t_1) \quad \text{, if } x \notin fv(t_2). \\
[y := t_2](\lambda_x.t_1) & \to_{abs3} & \lambda_z.([y := t_2](x\ z)t_1) \quad \text{, where } z \text{ is fresh and } x \in fv(t_2). \\
[y := t_3](t_1\ t_2) & \to_{app} & ([y := t_3]t_1)\ ([y := t_3]t_2)
\end{array} \tag{7}
$$

The propagation of a substitution through an abstraction is split into three rules, which ensure that free variables are not captured. In $abs1$, y does not occur free in $\lambda_y.t_1$, so the substitution $[y := t_2]$ has no effect. In $abs2$, the substitution is propagated inside the abstraction without renaming x, since x does not occur free in t_2. Finally, in $abs3$, x must be renamed to a fresh variable, since x occurs free in t_2. Note that [25] works modulo α-equivalence and therefore needs only one rule for abstraction: $[y := t_2](\lambda_x.t_1) \to \lambda_x.([y := t_2]t_1)$, i.e., it assumes that bound variables are renamed as needed to avoid capturing free variables.

To summarize the notation, $\to_x = \to_{var} \cup \to_{gc} \cup \to_{abs1} \cup \to_{abs2} \cup \to_{abs3} \cup \to_{app}$ and $\to_{lx} = \to_b \cup \to_x$.

Our goal is to prove that $\to_{lx}$ is confluent. To this end, we show that it satisfies the compositional Z property, which is a sufficient condition for confluence.

Our proof strategy is similar to that of [25], where the authors prove that the λ_x-calculus satisfies the compositional Z property. We take R in the Z property (Section 1) as the reduction relation $\to_{lx}$, and f as the composition of two functions, the complete permutation P and the complete development B, recursively defined as:

ROCQ

```
Fixpoint P (t : n_sexp) :=
  match t with
  | n_var x ⇒ n_var x
  | n_abs x t1 ⇒ n_abs x (P t1)
  | n_app t1 t2 ⇒ n_app (P t1) (P t2)
  | n_sub t1 x t2 ⇒ {x := (P t2)}(P t1)
  end.
```

ROCQ

```
Fixpoint B (t : n_sexp) :=
  match t with
  | n_var x ⇒ n_var x
  | n_abs x t1 ⇒ n_abs x (B t1)
  | n_app t1 t2 ⇒
      match t1 with
      | n_abs x t3 ⇒ {x := (B t2)}(B t3)
      | _ ⇒ n_app (B t1) (B t2)
      end
  | n_sub t1 x t2 ⇒ n_sub (B t1) x (B t2)
  end.
```

In what follows, we list the properties of P and B that are needed for the confluence proof, and sketch the proofs of the most interesting ones. First, $P\ t$ is a pure term, since P removes all explicit substitutions of t. This is proved by induction on t:

ROCQ

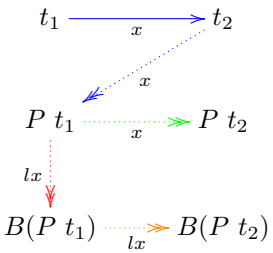
```
Lemma pure_P: ∀ t, pure (P t).
```

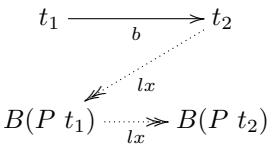
Similarly, B does not introduce explicit substitutions into a pure term:

ROCQ

```
Lemma pure_B: ∀ t, pure t → pure (B t).
```

As explained in Section 1, the compositional Z property decomposes the reduction relation into two subrelations and requires a Z-like property for each of them. For the λ_x-calculus, we have $\to_{lx} = \to_b \cup \to_x$, and the proof of the compositional Z property is summarized by the following diagrams:





(8)

The proof that the complete permutation function P is a Z function for the relation $\to_x$ (blue and green arrows) uses the fact that the explicit substitution implements the metasubstitution when its body is pure, i.e., when the body has no explicit substitution operator. This is formalized in the next lemma. Note that the hypothesis that *t1* is pure is needed because the λ_x-calculus does not have composition of explicit substitutions (cf. [19]), and hence the propagation of the outermost substitution $[y := t_2]$ in the term $[y := t_2]([x := t_1]t)$ must be delayed until the inner substitution $[x := t_1]$ is completely propagated.

ROCQ

```
Lemma pure_scx: ∀ t1 x t2, pure t1 → ([x := t2]t1) ⟶»x ({x := t2}t1).
```

Proof

The proof is by induction on the size of *t1*. The interesting case is when $t_1 = \lambda_y.t'_1$, where we need to prove that $[x := t_2](\lambda_y.t'_1) \longrightarrow\!\!\!\!\!\rightarrow_x \{x := t_2\}(\lambda_y.t'_1)$. There are two cases:

1. $(x = y)$: $[y := t_2](\lambda_y.t'_1) \longrightarrow_{abs1} \lambda_y.t'_1 = \{y := t_2\}(\lambda_y.t'_1)$.
2. $(x \neq y)$: Consider the following subcases:
   (a) If $y \in fv(t_2)$, let z be a fresh variable. Then
   $[x := t_2](\lambda_y.t'_1) \longrightarrow_{abs3} \lambda_z.[x := t_2]((y\ z)t'_1) \longrightarrow\!\!\!\!\!\rightarrow_x^{(i.h.)}$
   $\lambda_z.\{x := t_2\}((y\ z)t'_1) \overset{(\star)}{=}_\alpha \{x := t_2\}(\lambda_y.t'_1)$.
   (b) If $y \notin fv(t_2)$ then $[x := t_2](\lambda_y.t'_1) \longrightarrow_{abs2} \lambda_y.([x := t_2]t'_1) \longrightarrow\!\!\!\!\!\rightarrow_x^{(i.h.)}$
   $\lambda_y.(\{x := t_2\}t'_1) \overset{(\star)}{=}_\alpha \{x := t_2\}(\lambda_y.t'_1)$.
   where the $(\star)$ step (in both subcases) is intuitively simple but has a tricky proof since the definition (4) of metasubstitution systematically renames variables whenever it crosses an abstraction. □

ROCQ

```
Lemma refltrans_P: ∀ t, t ⟶»x (P t).
```

Proof

The proof is by induction on t. The interesting case is when t is an explicit substitution, i.e., $t = [x := t_2]t_1$. In this case, we have to show that

$$[x := t_2]t_1 \longrightarrow\!\!\!\!\!\rightarrow_x P([x := t_2]t_1)$$

where $P([x := t_2]t_1) = \{x := P\ t_2\}(P\ t_1)$ by definition of P. This is done as follows:

$$\dfrac{\dfrac{}{[x := t_2]t_1 \longrightarrow\!\!\!\!\!\rightarrow_x [x := P\ t_2](P\ t_1)}(\star) \qquad \dfrac{}{[x := P\ t_2](P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_x \{x := P\ t_2\}(P\ t_1)}(\star\star)}{[x := t_2]t_1 \longrightarrow\!\!\!\!\!\rightarrow_x \{x := P\ t_2\}(P\ t_1)}\ (\mathrm{T})$$

where (T) is a transitivity step, $(\star)$ follows from the induction hypothesis for t_1 and t_2, since $\longrightarrow\!\!\!\!\!\rightarrow_x$ is compatible with explicit substitutions, and $(\star\star)$ follows from Lemma *pure_scx*, since $P\ t_1$ is pure by Lemma *pure_P*. □

Therefore, Lemma *refltrans_P* proves the blue arrows in (8). The green arrow in (8) is given by Lemma *scx_P* below, which shows that $P\ t_1 =_\alpha P\ t_2$. The main auxiliary results needed for *scx_P* are:

ROCQ

```
Lemma aeq_swap_P: ∀ t x y, (P (swap x y t)) =α (swap x y (P t)).

Lemma aeq_P: ∀ t1 t2, t1 =α t2 → (P t1) =α (P t2).
```

Lemma *aeq_swap_P* is proved by induction on t, and Lemma *aeq_P*, by induction on the derivation $t1 =\alpha\ t2$. We omit their proofs to save space.

ROCQ

```
Lemma scx_P: ∀ t1 t2, (t1 →x t2) → (P t1) =α (P t2).
```

Proof

The proof is by induction on the reduction $(t1 \to x\ t2)$. The non-trivial case is when $t1 \to_{abs3} t2$. In this case,

$$t1 = [y := t_2'](\lambda_x.t_1') \to_{abs3} \lambda_z.[y := t_2'](x\ z)t_1' = t2,$$

where $x \neq y$ and z is a fresh variable. We proceed as follows:

$$\dfrac{\dfrac{\dfrac{}{\{y := P\ t_2'\}\lambda_x.(P\ t_1') =_\alpha \lambda_z.\{y := P\ t_2'\}((x\ z)P\ t_1')}(\diamond)}{\{y := P\ t_2'\}\lambda_x.(P\ t_1') =_\alpha \lambda_z.\{y := P\ t_2'\}P\ ((x\ z)t_1')}\ (aeq_swap_P)}{P\ ([y := t_2'](\lambda_x.t_1')) =_\alpha P\ (\lambda_z.[y := t_2'](x\ z)t_1'))}\ (\mathrm{DEF.}\ P)$$

where ($\diamond$) is an application of Lemma *aeq_m_subst_abs_neq*. □

The red arrow in (8) is given by Lemma *pure_refltrans_B*:

ROCQ

```
Lemma pure_refltrans_B: ∀ t, pure t → t ⟶»lx (B t).
```

Proof

The proof is by induction on the size of the term t. The interesting case is when t is an application, i.e., $t = t_1\ t_2$, and we have to show that $t_1\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(t_1\ t_2)$. If t_1 is not an abstraction, then $B(t_1\ t_2) = B(t_1)\ B(t_2)$ and we are done by the induction hypothesis. If t_1 is an abstraction, say $t_1 = \lambda_x.t_1'$, then we have to show that

$$(\lambda_x.t_1')\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} B((\lambda_x.t_1')\ t_2) = \{x := (B\ t_2)\}(B\ t_1')$$

whose derivation is as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\lambda_x.t_1' \longrightarrow\!\!\!\!\!\rightarrow_{lx} \lambda_x.B(t_1')}\ (\text{I.H.})}{\lambda_x.t_1' \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(\lambda_x.t_1')}\ (\text{DEF. } B)}{(\lambda_x.t_1')\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} (B(\lambda_x.t_1'))\ t_2}\ (\nabla) \qquad \dfrac{}{(B(\lambda_x.t_1'))\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t_1')}\ (\star)}{(\lambda_x.t_1')\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t_1')}\ (\text{T})$$

where (∇) is an application of Lemma refltrans_n_app_left_lx and ($\star$) is proved as follows:

$$\dfrac{(\star\star) \qquad \dfrac{}{(B(\lambda_x.t_1'))\ (B\ t_2) \longrightarrow\!\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t_1')}\ (\dagger)}{(B(\lambda_x.t_1'))\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} \{x := (B\ t_2)\}(B\ t_1')}\ (\text{T})$$

where (T) is a transitivity step and ($\star\star$) is given by:

$$(\diamond)\ \dfrac{\dfrac{}{t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} (B\ t_2)}\ (\text{I.H.})}{(B(\lambda_x.t_1'))\ t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} (B(\lambda_x.t_1'))\ (B\ t_2)}$$

where ($\diamond$) is an application of Lemma refltrans_n_app_right_lx. Finally, ($\dagger$) is given by the reduction:

$$(B(\lambda_x.t_1'))\ (B\ t_2) \to_b [x := B\ t_2](B\ t_1') \overset{(\star\star\star)}{\longrightarrow\!\!\!\!\!\rightarrow_{lx}} \{x := (B\ t_2)\}(B\ t_1')$$

where ($\star\star\star$) follows from Lemma pure_scx, since $B\ t_1'$ is pure by Lemma pure_B. □

In order to complete the proof of the left diagram in (8), we need to show that $B(P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_2)$ (orange arrow). By Lemma scx_P, $P\ t_1 =_\alpha P\ t_2$, and we conclude because B is stable under α-equivalence, as shown by the following lemma:

ROCQ

```
Lemma aeq_B: ∀ t1 t2, t1 =α t2 → (B t1) =α (B t2).
```

We now turn to the right diagram in (8): given $t_1 \to_b t_2$, we want to prove that $t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_2)$. The first part is Lemma *step_b_refltrans_lx_B_P* below, which is proved by induction on the reduction $t_1 \to_b t_2$; its tricky step is to prove that

$$\{x := B(P\ t_2)\}(B(P\ t_1)) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(\{x := P\ t_2\}(P\ t_1)) \tag{9}$$

To prove it, we use the fact that $\to_{lx}$ simulates $\to_\beta$, which is now defined as in (2), but over *n_sexp* terms.

On pure terms, every β-step can be simulated by a $\longrightarrow\!\!\!\!\!\rightarrow_{lx}$-reduction. This allows us to reduce the proof of (9) to a statement about β-reduction:

ROCQ

```
Lemma pure_beta_lx: ∀ t1 t2, pure t1 → (t1 →β t2) → (t1 ⟶»lx t2).
```

Since this lemma extends to $\longrightarrow\!\!\!\!\!\rightarrow_\beta$ by a straightforward induction on the number of steps, it remains to show that

$$\{x := B(P\ t_2)\}(B(P\ t_1)) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := P\ t_2\}(P\ t_1))$$

which is the statement of the following lemma:

ROCQ

```
Lemma pure_m_subst_refltrans_beta_B: ∀ t1 t2 x, pure t1 → pure t2 →
                                     ({x := B t2} B t1) ⟶»β (B ({x := t2} t1)).
```

Proof

The proof is by induction on the size of *t1*. The first interesting case is when *t1* is an application $t_{11}\ t_{12}$, where we need to prove that

$$\{x := B\ t_2\}(B(t_{11}\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}(t_{11}\ t_{12}))$$

We proceed by case analysis on t_{11}. If t_{11} is the variable x, then the goal becomes

$$(B\ t_2)\ (\{x := B\ t_2\}(B\ t_{12})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B\ (t_2\ (\{x := t_2\}t_{12}))$$

which is proved by the reduction

$$(B\ t_2)\ (\{x := B\ t_2\}(B\ t_{12})) \xrightarrow[\beta]{\text{(i.h.)}} (B\ t_2)\ (B(\{x := t_2\}t_{12})) \xrightarrow[\beta]{(\diamond)} B\ (t_2\ (\{x := t_2\}t_{12}))$$

where the first step applies the induction hypothesis to t_{12} in the argument of the application (Lemma refltrans_n_app_right), and $(\diamond)$ is Lemma n_app_refltrans_beta_B.

The other non-trivial case is when t_{11} is an abstraction $\lambda_y.t'_{11}$ (the remaining cases follow directly from the induction hypothesis). By definition of B, the goal is

$$\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := t_2\}((\lambda_y.t'_{11})\ t_{12}))$$

If $x = y$, the result follows from Lemma aeq_double_m_subst and the induction hypothesis applied to t_{12}. If $x \neq y$, let z be a fresh name. Then

$$\begin{aligned}
&\{x := B\ t_2\}(\{y := B\ t_{12}\}(B\ t'_{11})) \\
&\quad =_\alpha \{z := \{x := B\ t_2\}(B\ t_{12})\}(\{x := B\ t_2\}((y\ z)(B\ t'_{11}))) \\
&\quad \xrightarrow[\beta]{(\star)} \{z := B(\{x := t_2\}t_{12})\}(B(\{x := t_2\}((y\ z)t'_{11}))) \\
&\quad =_\alpha B(\{x := t_2\}((\lambda_y.t'_{11})\ t_{12}))
\end{aligned}$$

where the first α-equivalence step renames y to z and applies the Substitution Lemma (m_subst_lemma) formalized in [20]; $(\star)$ applies the induction hypothesis to t_{12} and to $(y\ z)t'_{11}$, which has the same size as t'_{11} (Lemma pure_refltrans_beta_m_subst_compat); and the last step follows from the definition of B and Lemma aeq_m_subst_abs_neq. □

We can now prove the first part of the right diagram in (8):

ROCQ

```
Lemma step_b_refltrans_lx_B_P: ∀ t1 t2, (t1 →b t2) → (t2 ⟶lx (B(P t1))).
```

Proof

The interesting case is when t_1 is an explicit substitution, that is, either

$$t_1 = [x := t_{12}]t_{11} \to_b [x := t_{12}]t'_{11} = t_2, \text{ with } t_{11} \to_b t'_{11}$$

or

$$t_1 = [x := t_{12}]t_{11} \to_b [x := t'_{12}]t_{11} = t_2, \text{ with } t_{12} \to_b t'_{12}.$$

Both cases are similar, so we consider only the first one. Since $P([x := t_{12}]t_{11}) = \{x := P\ t_{12}\}(P\ t_{11})$, the proof proceeds as follows:

$$\dfrac{\dfrac{\dfrac{(\text{I.H.})\ \dfrac{}{t'_{11} \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_{11})} \qquad \dfrac{}{t_{12} \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_{12})}\ (\star)}{[x := t_{12}]t'_{11} \longrightarrow\!\!\!\!\!\rightarrow_{lx} [x := B(P\ t_{12})](B(P\ t_{11}))}\ (\text{SUB})}{[x := t_{12}]t'_{11} \longrightarrow\!\!\!\!\!\rightarrow_{lx} B([x := P\ t_{12}](P\ t_{11}))}\ (\text{DEF. } B) \qquad (\star\star)}{[x := t_{12}]t'_{11} \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}\ (\text{T})$$

where (sub) is Lemma refltrans_n_sub_lx, (T) is transitivity, and $(\star)$ is the composition of $t_{12} \longrightarrow\!\!\!\!\!\rightarrow_x P\ t_{12}$ (Lemma refltrans_P) with $P\ t_{12} \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_{12})$ (Lemma pure_refltrans_B, since $P\ t_{12}$ is pure by Lemma pure_P). The reduction $(\star\star)$, that is, $B([x := P\ t_{12}](P\ t_{11})) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))$, is proved as follows:

$$\dfrac{\dfrac{(\star\star\star) \qquad (\dagger)}{[x := B(P\ t_{12})](B(P\ t_{11})) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}\ (\text{T})}{B([x := P\ t_{12}](P\ t_{11})) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(\{x := P\ t_{12}\}(P\ t_{11}))}\ (\text{DEF. } B)$$

where $(\star\star\star)$ is the reduction $[x := B(P\ t_{12})](B(P\ t_{11})) \longrightarrow\!\!\!\!\!\rightarrow_{lx} \{x := B(P\ t_{12})\}(B(P\ t_{11}))$, an x-reduction given by Lemma pure_scx, since $B(P\ t_{11})$ is pure (Lemmas pure_B and pure_P), and $\rightarrow_x \subseteq \rightarrow_{lx}$. The reduction $(\dagger)$ is the tricky part of the proof: Lemma pure_m_subst_refltrans_beta_B gives $\{x := B(P\ t_{12})\}(B(P\ t_{11})) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(\{x := P\ t_{12}\}(P\ t_{11}))$, and this β-reduction is simulated by an lx-reduction because $\rightarrow_{lx}$-steps simulate β-steps on pure terms (Lemma pure_beta_lx). □

Finally, the last part of the right diagram in (8) is the following lemma:

ROCQ

```
Lemma refltrans_lx_B_P: ∀ t1 t2, (t1 →b t2) → (B (P t1)) ⟶→lx (B (P t2)).
```

Proof

The proof goes through β-reduction, and it is summarized by the following derivation:

$$\dfrac{\dfrac{\dfrac{t_1 \rightarrow_b t_2}{P\ t_1 \longrightarrow\!\!\!\!\!\rightarrow_\beta P\ t_2}\ (a)}{B(P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_\beta B(P\ t_2)}\ (b)}{B(P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_2)}\ (c)$$

where

– (a) is Lemma b_refltrans_beta_P: P maps each b-step to a β-reduction. In fact, P turns every explicit substitution into a meta-

substitution, so the b-step $(\lambda_x.t)\ u \to_b [x := u]t$ becomes the β-step $(\lambda_x.P\ t)\ (P\ u) \to_\beta \{x := P\ u\}(P\ t)$;
- (b) is Lemma refltrans_beta_B: B preserves β-reductions that start from a pure term, and $P\ t_1$ is pure by Lemma pure_P;
- (c) is Lemma pure_refltrans_beta_lx: on pure terms, every β-reduction is simulated by an lx-reduction, and $B(P\ t_1)$ is pure by Lemmas pure_B and pure_P.

□

The diagrams in (8) are formalized as the predicate *Z_comp_aeq* used in the next lemma:

ROCQ

```
Lemma lambda_x_Z_comp_aeq: Z_comp_aeq lambdax.
```

Proof

We take $R_1 = \to_x$, $R_2 = \to_b$, $f_1 = P$ and $f_2 = B$, and check the conditions of *Z_comp_aeq* against the diagrams in (8):

- $\to_{lx} = \to_x \cup \to_b$, by the definition of $\to_{lx}$;
- left diagram: if $t_1 \to_x t_2$, then $P\ t_1 =_\alpha P\ t_2$ by Lemma scx_P, so the green arrow is an α-equivalence, and then $B(P\ t_1) =_\alpha B(P\ t_2)$ (orange arrow) by Lemma aeq_B. The blue arrows follow from $t \longrightarrow\!\!\!\!\!\rightarrow_x P\ t$, for every t (Lemma refltrans_P): we get $t_2 \longrightarrow\!\!\!\!\!\rightarrow_x P\ t_2 =_\alpha P\ t_1$. The red arrow follows from Lemma pure_refltrans_B, since $P\ t_1$ is pure (Lemma pure_P);
- right diagram: if $t_1 \to_b t_2$, then Lemmas step_b_refltrans_lx_B_P and refltrans_lx_B_P give $t_2 \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_1)$ and $B(P\ t_1) \longrightarrow\!\!\!\!\!\rightarrow_{lx} B(P\ t_2)$, respectively. That is, $B \circ P$ has the weak Z property for $\to_b$ with respect to $\to_{lx}$;
- finally, $B \circ P$ is compatible with α-equivalence, by Lemmas aeq_P and aeq_B.

□

Therefore, we can conclude that the reduction $\to_{lx}$ satisfies the compositional Z property modulo α-equivalence. In [11], we proved that the compositional Z property implies the Z property, which in turn implies confluence. The versions of these results modulo α-equivalence give the confluence of $\to_{lx}$:

ROCQ

```
Theorem lambda_x_is_confluent: Confl lambdax.
Proof.
  apply Z_prop_aeq_implies_Confl.
  apply Z_comp_aeq_implies_Z_prop_aeq.
  apply lambda_x_Z_comp_aeq.
```

```
Qed.
```

3 Related Work and Conclusion

Related Work. The Z property was introduced by van Oostrom [28] and Dehornoy and van Oostrom [12] as a simple, self-contained alternative to the classical Tait-Martin-Löf technique for proving confluence [17,26,27], and it has since been used as a benchmark for confluence tools and formalizations [13,34]. Nakazawa and Fujita [25] proposed the compositional Z property, an extension tailored to calculi whose reduction relation splits naturally into an initiating step and a substitution calculus that completes it, and used it to give a pencil-and-paper confluence proof for a calculus with explicit substitutions close to the λ_x-calculus studied here. The present paper can be seen as a full mechanization, and a nominal reformulation, of that proof strategy.

Our development directly builds on two previous formalizations by the author and collaborators: [11], which formalizes the Z and compositional Z properties, and [20], which extends the classical Substitution Lemma to calculi with explicit substitutions. Related nominal infrastructures have been developed in other proof assistants, notably Nominal Isabelle [33] and a nominal library for Agda [14]; unlike these, our formalization does not rely on a dedicated nominal package or on quotient types, but implements swapping and α-equivalence directly as inductive definitions over raw terms, in the same spirit as other machine-checked accounts of the Barendregt variable convention [8,32,2].

Conclusion. We presented a nominal formalization, in the Rocq proof assistant (version 9.1.1), of a framework for proving confluence via the (compositional) Z property, together with the supporting theory of α-equivalence, swapping, and capture-avoiding metasubstitution for nominal terms with an explicit substitution operator. As an application of this framework, we formalized the λ_x-calculus and proved that its reduction relation $\to_{lx}$ is confluent, by exhibiting the complete permutation function P and the complete development function B as witnesses of the compositional Z property. To the best of our knowledge, this is the first fully mechanized, nominal proof of confluence for a calculus with explicit substitutions via the (compositional) Z property. The whole development is freely available (`https://github.com/flaviodemoura/Zprop-nominal.git`) and comprises around 3 500 lines of Rocq code, organized in 150 lemmas and theorems, which can serve as a reusable toolkit for confluence proofs of other calculi with explicit substitutions.

As future work, we intend to extend the framework with metavariables, which play an important role in the study of unification, and to apply it to other calculi with explicit substitutions from the long list surveyed in the introduction [1,30,4,9,24,18,6,10,19], including calculi that support composition of substitutions, a feature the λ_x-calculus deliberately avoids. We also plan a systematic, quantitative comparison between the nominal formalization presented here and

the locally nameless formalizations [7] it builds upon, in terms of proof size, automation effort, and robustness to changes in the calculus under study.

Finally, the formalization of the Substitution Lemma in [20] relies on an axiom (*Eq_implies_equality*) stating that α-equivalent terms have (Leibniz) equal sets of free variables, whereas we prove this property only up to set equality (aeq_fv_nom). The axiom is used to rewrite in the body of a `let` expression that chooses a fresh name with `atom_fresh`. A pull request to Rocq (`rocq-prover/rocq#20985`) would allow us to remove it, but it has not been merged yet. Alternatively, we plan to use an equivalent notion of α-equivalence in which the free variables are implemented as lists, instead of sets.

References

1. Abadi, M., Cardelli, L., Curien, P.L., Lévy, J.J.: Explicit Substitutions. Journal of Functional Programming **1**(4), 375–416 (1991). https://doi.org/10.1017/S0956796800000186
2. Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S.: Engineering Formal Metatheory. ACM SIGPLAN Notices **43**(1), 3 (2008). https://doi.org/10.1145/1328897.1328443
3. Barendregt, H.P.: The Lambda Calculus: Its Syntax and Semantics. No. v. 103 in Studies in Logic and the Foundations of Mathematics, North-Holland ; Sole distributors for the U.S.A. and Canada, Elsevier Science Pub. Co, Amsterdam ; New York : New York, N.Y, rev. ed edn. (1984)
4. Benaissa, Z.E.A., Briaud, D., Lescanne, P., Rouyer-Degli, J.: $\lambda\nu$, a calculus of explicit substitutions which preserves strong normalisation. Journal of Functional Programming **6**(5), 699–722 (Sep 1996). https://doi.org/10.1017/S0956796800001945
5. Bloo, R., Rose, K.: Preservation of Strong Normalisation in Named Lambda Calculi with Explicit Substitution and Garbage Collection. In: CSN-95: COMPUTER SCIENCE IN THE NETHERLANDS. pp. 62–72 (1995)
6. Bloo, R., Geuvers, H.: Explicit Substitution: On the Edge of Strong Normalization. Theoretical Computer Science **211**(1-2), 375–395 (1999). https://doi.org/10.1016/s0304-3975(97)00183-7
7. Charguéraud, A.: The Locally Nameless Representation. Journal of Automated Reasoning pp. 1–46 (2011). https://doi.org/10.1007/s10817-011-9225-2
8. Copello, E., Szasz, N., Tasistro, Á.: Machine-checked Proof of the Church-Rosser Theorem for the Lambda Calculus Using the Barendregt Variable Convention in Constructive Type Theory. Electronic Notes in Theoretical Computer Science **338**, 79–95 (Oct 2018). https://doi.org/10.1016/j.entcs.2018.10.006
9. Curien, P.L., Hardin, T., Lévy, J.J.: Confluence Properties of Weak and Strong Calculi of Explicit Substitutions. Journal of the ACM **43**(2), 362–397 (1996). https://doi.org/10.1145/226643.226675
10. David, R., Guillaume, B.: A lambda-calculus with explicit weakening and explicit substitution. Mathematical Structures in Computer Science **11**(1), 169–206 (2001). https://doi.org/10.1017/S0960129500003224
11. de Moura, F.L.C., Rezende, L.O.: A formalization of the (compositional) Z property. In: Fifth Workshop on Formal Mathematics for Mathematicians (2021)

12. Dehornoy, P., van Oostrom, V.: Z, proving confluence by monotonic single-step upperbound functions. Logical Models of Reasoning and Computation (LMRC-08) p. 85 (2008)
13. Felgenhauer, B., Nagele, J., van Oostrom, V., Sternagel, C.: The Z property. Archive of Formal Proofs (Jun 2016)
14. Gabbay, M.J., Melkonian, O.: Nominal techniques as an Agda library (Mar 2026). https://doi.org/10.48550/arXiv.2603.03968
15. Gabbay, M.J., Pitts, A.M.: A New Approach to Abstract Syntax with Variable Binding. Formal Aspects of Computing **13**(3-5), 341–363 (Jul 2002). https://doi.org/10.1007/s001650200016
16. Guillaume, B.: The λ_{se}-calculus does not preserve strong normalisation. Journal of Functional Programming **10**(4), 321–325 (Jul 2000). https://doi.org/10.1017/S0956796800003695
17. Honda, Y., Nakazawa, K., Fujita, K.e.: Confluence Proofs of Lambda-Mu-Calculi by Z Theorem. Studia Logica **109**(5), 917–936 (Oct 2021). https://doi.org/10.1007/s11225-020-09931-0
18. Kamareddine, F., Ríos, A.: Extending a λ-calculus with explicit substitution which preserves strong normalisation into a confluent calculus on open terms. Journal of Functional Programming **7**(4), 395–420 (Jul 1997). https://doi.org/10.1017/S0956796897002785
19. Kesner, D.: A Theory of Explicit Substitutions with Safe and Full Composition. Logical Methods in Computer Science **Volume 5, Issue 3**, 816 (Jul 2009). https://doi.org/10.2168/LMCS-5(3:1)2009
20. Lima, M.J.D., de Moura, F.L.C.: A Formalized Extension of the Substitution Lemma in Coq. EPTCS **389**, 80–95 (2023). https://doi.org/10.4204/EPTCS.389.7
21. Lins, R.: A new formula for the execution of categorical combinators. 8th Conference on Automated Deduction (CADE) **volume 230 of LNCS**, 89–98 (1986)
22. Lins, R.: Partial categorical multi-combinators and Church Rosser theorems. Tech. Rep. 7/92, Computing Laboratory, University Kent at Canterbury (May 1992)
23. Mellies, P.A.: Typed λ-calculi with explicit substitutions may not terminate. In: Goos, G., Hartmanis, J., Van Leeuwen, J., Dezani-Ciancaglini, M., Plotkin, G. (eds.) Typed Lambda Calculi and Applications, vol. 902, pp. 328–334. Springer Berlin Heidelberg, Berlin, Heidelberg (1995). https://doi.org/10.1007/BFb0014062
24. Muñoz, C.A.: Confluence and Preservation of Strong Normalisation in an Explicit Substitutions Calculus. In: Proceedings, 11th Annual IEEE Symposium on Logic in Computer Science, New Brunswick, New Jersey, USA, July 27-30, 1996. pp. 440–447 (1996). https://doi.org/10.1109/LICS.1996.561460
25. Nakazawa, K., Fujita, K.e.: Compositional Z: Confluence Proofs for Permutative Conversion. Studia Logica **104**(6), 1205–1224 (2016). https://doi.org/10.1007/s11225-016-9673-0
26. Nakazawa, K., Fujita, K.e.: Z for call-by-value. In: 6th International Workshop on Confluence (IWC 2017). pp. 57–61 (2017)
27. Nakazawa, K., Fujita, K.e., Imagawa, Y.: Z property for the shuffling calculus. Mathematical Structures in Computer Science pp. 1–13 (Jan 2023). https://doi.org/10.1017/S0960129522000408
28. van Oostrom, V.: Z - Draft: For Your Mind Only (2007)
29. Rose, K.: Explicit cyclic substitutions. In: Rusinowitch, M., Rémy, J.L. (eds.) 3rd International Workshop on Conditional Term Rewriting Systems (CTRS). vol. 656, pp. 36–50. Springer-Verlag (1992)

30. Rose, K.H., Bloo, R., Lang, F.: On Explicit Substitution With Names. J Autom Reasoning **49**(2), 275–300 (2011). https://doi.org/10.1007/s10817-011-9222-5
31. Urban, C., Pitts, A.M., Gabbay, M.J.: Nominal Unification. Theoretical Computer Science **323**(1-3), 473–497 (2004). https://doi.org/10.1016/j.tcs.2004.06.016
32. Urban, C., Berghofer, S., Norrish, M.: Barendregt's Variable Convention in Rule Inductions. In: Pfenning, F. (ed.) Automated Deduction – CADE-21. pp. 35–50. Springer, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-73595-3_4
33. Urban, C., Kaliszyk, C.: General Bindings and Alpha-Equivalence in Nominal Isabelle. Logical Methods in Computer Science **Volume 8, Issue 2** (Jun 2012). https://doi.org/10.2168/LMCS-8(2:14)2012
34. van Oostrom, V.: Z; syntax-free developments. In: Kobayashi, N. (ed.) 6th International Conference on Formal Structures for Computation and Deduction (FSCD 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 195, pp. 24:1–24:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). https://doi.org/10.4230/LIPIcs.FSCD.2021.24